Adol Awan 30237232
Kai Christie 30210637
Mamoon Khan 30223013
Farnood Mojahedzadeh 30176848
Vihan Shah 30114292
Hassan Waraych 30173507

I. Introduction

StudyDigest is a student-focused application designed to streamline academic review by transforming raw notes into concise summaries and automatically generated practice questions. Many students struggle with information overload, inefficient study habits, and difficulty identifying key concepts. This system addresses these challenges by leveraging Large Language Models (LLMs) to convert uploaded notes into structured, digestible study material.

This SRS document defines the functional and non-functional requirements for StudyDigest. It will guide the development team, inform stakeholders, and serve as a reference for quality assurance activities.

StudyDigest allows users to upload course notes in text or PDF format. The system processes the content using an LLM to produce:

- A concise summary of the material
- A set of review questions (multiple-choice, short-answer, or conceptual prompts)

Intended Audience:

1. **Students (end-users):** Primary users who upload notes and receive summaries/questions.
2. **Project Managers:** Oversee development progress and ensure alignment with project goals.
3. **Developers:** Implement the system based on the requirements in this document.
4. **Quality Assurance Team:** Validate that the system meets functional and non-functional requirements.
5. **Teaching Assistants / Instructors:** Evaluate the project deliverables.

Definitions, Acronyms, and Abbreviations:

- **LLM:** Large Language Model
- **UI:** User Interface
- **API:** Application Programming Interface
- **PDF:** Portable Document Format
- **Summary:** A condensed version of the uploaded notes

- **Review Questions:** Automatically generated questions based on the uploaded content

II. Overall Description

The system will operate as a standalone web application. It does not replace learning management systems but complements them by improving study efficiency and comprehension.

User Classes:

1. **Student:** Students are the primary users. Students upload notes and are given the option to download generated summaries and review questions. Students are responsible for using the system ethically and responsibly.
2. **System Administrator:** Administrators are responsible for managing the users. System administrators will have a different interface and will not have the functionalities students will have.

The system will run on a web browser and will support modern web browsers such as Chrome, Safari, Edge, Firefox, and more. It will not require an installation.

Due to cost constraints, our project may have limited API calls as well as storage limits. Our system will use a relational database management system that uses SQL to store and manage data in order to align with team expertise.

Our project explicitly assumes users have internet access and that users upload readable PDFs. Our project also assumes our selected LLM API will be available. Our project implicitly assumes the LLM will produce mostly accurate summaries. Our project depends on our backend server, database connection and frontend framework not failing. Finally, our project depends on the LLM API provider to operate.

III. System Features

Feature F1: User Account Management
Description: Allows users to create accounts, log in, and manage their profile.
Priority: Low
Functional Requirements:

1. The system shall allow users to register using an email and password.
2. The system shall allow users to log in securely.
3. The system shall allow users to reset their password.
4. The system should generate question and summary

Feature F2: Note Upload
Description: Users upload notes in supported formats.
Priority: High
Functional Requirements:

1. The system shall allow users to upload notes in PDF or text format.
2. The system shall validate file size and format.
3. The system shall notify users if the upload fails.

Feature F3: View Study Material
Description: View and Download System Responses in supported formats
Priority: High
Functional Requirements:

1. The system shall allow users to download the generated summaries and review questions in text and csv format.
2. The system shall allow users to view the generated content without downloading

Feature F4: Generate Study Material
Description: Leveraging LLMs to convert uploaded notes into digestible study material
Priority: High
Functional Requirements:

1. The system should create a summary of the uploaded notes.
2. The system should generate a review question based on the uploaded material
3. The system should allow user to select wich type of review question to generate
4. The system should allow user to regenerate the material if the user is not satisfied

Feature F5: User Management
Description: Admins can manage (view, modify, delete, etc) user accounts
Priority: Low
Functional Requirements:

1. The system shall allow admins to view all users accounts
2. The system shall allow admins to modify the details of a users' account when asked to do so
3. The system shall allow admins to delete user accounts if necessary

Feature F5: View Previous Summaries
Description: Users are allowed to view previous summaries generated from prior uploads
Priority: Medium
Functional Requirements:

1. The system shall allow users to select and view generated summaries from prior uploads

IV. External Interface Requirements

A. User Interface Requirements

1) Web Application Interface
    - Responsive web design supporting desktop (1920 x 1080 minimum)
    - Maximum page load time: 3 seconds
    - Dashboard showing upload history, recently generated summaries and quick actions
    - Progress indicators for file upload and content generation processes
    - Intuitive file upload interface supporting drag and drop

The main page that every user is directed to is the home page which will contain a description of the application. A sign up and log in option will be available.

There will be two main forms users will access on our system. All users will interact with a “sign up” form as well as a “log in” form. After signing in, students will have access to the student main menu interface which will allow students to upload a new document. Alternatively, students can select a previously uploaded document. After selecting one of the two options, students will be directed to the main study digest interface which will contain the summary and review questions. Additionally, this interface will give the students the option to download the content in the supported formats.

After signing in, the system administrators will be directed to the admin main menu. This main menu will display the users to the admin and allow the admin to delete or make modifications to the users.

See our sketch in figure 1 below.

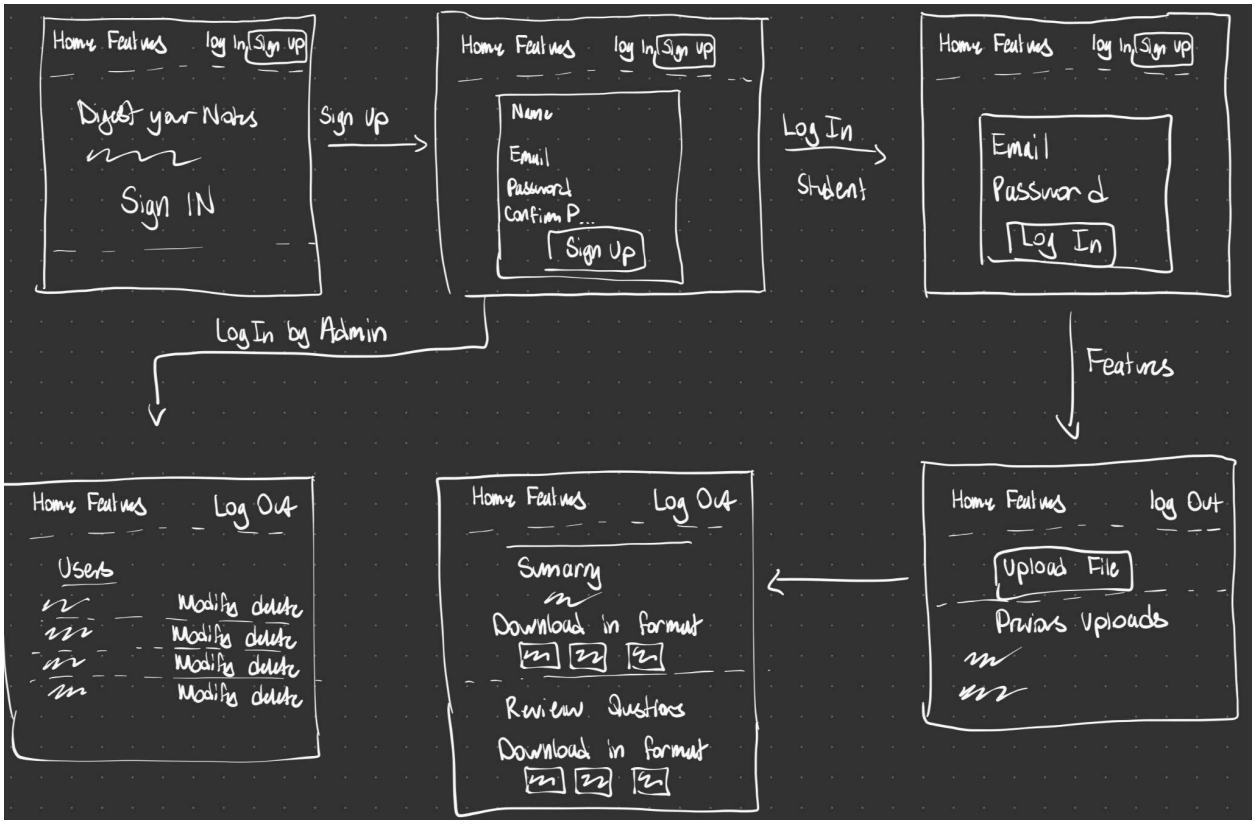

Figure 1: Sketch of interfaces layout and flow

B. Software Interface Requirements

1) LLM API Integration
   - RESTful API connection to LLM service (OpenAI)
   - Secure API key management and rotation capability
   - Error handling for API quota exceeded, service unavailable, invalid responses
   - Timeout handling (>60 seconds) for generation requests

2) Database Interface
   - SQL database connection
   - Backup and recovery mechanisms

C. Communication Interface Requirements

1) HTTPS Protocols
   - All communications over HTTPS
   - RESTful API architecture for client-server communication
   - JSON data format for request/response payloads

2) API Endpoints Structure
    - **/auth** - Authentication operations
    - **/upload -** File upload handling
    - **/generate** - Content generation requests
    - **/download** - Generated content retrieval
    - **/user** - User profile management

D. File Format Interface Requirements

1) Input Format Support
    - PDF files (text-extractable)
    - Plain text (.txt, .md)
    - Character encoding (UTF-8)
2) Export Format Support
    - Downloadable formats (PDF, DOCX, TXT, Markdown)
    - In-app HTML rendering for preview
    - Document generation libraries integration

E. Authentication & Security Interface Requirements

1) Authentication Services
    - JWT token-based authentication
    - Password hashing using Argon2
    - Session management with secure cookie handling

V. Non-Functional Requirements

Our system's non-functional requirements include the following:

- **Performance:** The system shall generate a summary and review questions within 60 seconds for 95% of text-based PDF requests
- **Security:** The system shall protect user data using encryption
- **Usability:** The user interface should be intuitive and easy for all users to follow with no training
- **Reliability:** The system should be available 99.9% of the time
- **Maintainability:** The system shall use a modular architecture such that features can be modified and added without affecting unrelated components
- **Accessibility:** The system will follow WCAG 2.1 accessibility guidelines to support screen-reader compatibility